



Nobody cheats in my class!

Hacettepe Computer Engineering - Spring 2026

BBM234 - MIPS Project

HABABAM SINIFI

The Chaos Class Tries to Cheat



Hababam Sınıfı - the Chaos Class Tries to Cheat

Külyutmaz Necmi enters the Hababam Sınıfı and announces that he's going to test the students. He dictates a couple of ridiculously hard questions and starts shouting:

“No talking, no looking to the right, no looking to the left, no leaning under the desks. I'll take your exam papers. Get ready to start. Five, four, three, two, one, zero, go!”



And as he walks over the student desks, he continues shouting frantically:

“Don't you dare try to cheat! In all of my thirty years of teaching, not a single student managed to cheat EVER. Bana kül yutturamazsınız!”

The situation in the classroom was pretty tense and at the very least morbid (to get a better idea [see this](#)

[footage](#)).

As the students have been coming up with more and more sophisticated cheating techniques, while the poor Külyutmaz has been getting older with declining fitness and agility, not to mention the eyesight, he finally understood that it was high time he found some alternative ways of catching the cheaters.

For this exam he came up with a brilliant idea of utilizing a computer for those purposes, and little did the cunning Hababam Sınıfı students know what was about to hit them.

The school had one SGI Indigo workstation computer which runs MIPS R3000, and all he needed were some MIPS programmers to help him realize his clever plan.

You, BBM234 students, are going to be “the good guys” and help Külyutmaz Necmi catch the cheaters.



MIPS Simulators

In this project, you will learn how to write and simulate **MIPS** code. You can either use **MARS** or **QtSpim** MIPS simulator for this project.

Download **MARS** from <https://computerscience.missouristate.edu/mars-mips-simulator.htm> and install it to your computer. The second simulator you can use is **QtSpim**, which can be downloaded from the following link:

<http://spimsimulator.sourceforge.net/>

There are several tutorials for MARS and QtSpim, some of which are listed below. If you can find better ones, please share on Piazza.

- <https://www.youtube.com/watch?v=r8WcV7AiLXs>
- https://cs.slu.edu/~fritts/CSCI140_F12/schedule/MARS%20Tutorial.pdf
- <https://ecs-network.serv.pacific.edu/past-courses/2014-fall-ecpe-170/tutorials/qtspim-tutorial>

How to Write and Simulate Your Code

- You should use a text editor and write your MIPS program first. If you are using MARS simulator, you can edit your MIPS programs directly inside the simulator (recommended).
- Save your program with an extension `.asm`
- Open the simulator of your choice and load your program.
- Assemble and run your program.
- Your results will be either in registers or in memory (data segment) based on your program output.

Terminating MIPS Programs on Simulators

Keep in mind that your main function will be called by the simulator as you call any other function in your code (think of it as THE ACTUAL MAIN function is provided by the simulator, while your code is just some other function that is called). Therefore, there are some rules you need to follow, namely, you need to consider what you need to do to make sure the return address is retrieved properly when your code terminates.

You have two options for termination. The first one: before you type any code, you should first save the return address `$ra` on stack, and as the last step restore it by executing `jr $ra`. An example is shown in Figure 1 below.


```
.text
.globl main
main:
    addi $sp, $sp, -4    # make room on stack to save $ra
    sw $ra, 0($sp)      # Save $ra on the stack
    # Your code here....
done:
    lw $ra, 0($sp)      # restore $ra
    addi $sp, $sp, 4    # deallocate stack space
    jr $ra              # return
```

Figure 1: Sample termination of a MIPS program in simulators.

```
.text
.globl main
main:
    # Your code here....
done:
    li $v0, 10
    syscall              # Execute the 'exit' syscall
```

Figure 2: Another way to terminate a MIPS program in simulators.

The second option: you can execute the ‘exit’ syscall as shown in the Figure 2.



Mission: Detecting Cheaters

Considering how Hababam Sınıfı students have proven time and time again how cunning, deceitful, and resourceful they can be, special precautions had to be taken to hide the intention of this cheating detection program, in case they gain unauthorized access to the school's computer and snoop around the files. If they were to understand what the data meant and what its purpose is, they could easily change it to their benefit to avoid detection again. Therefore, Külyutmaz Necmi decided to keep all the data about the students and their scores using covert methods as described in the following paragraphs. *"They'll never figure this out!"* he muttered to himself as he came up with his plan over a cup of çay at the teachers' lounge.

The students wouldn't be stored by their names, but special IDs would be assigned to each student, known only to teacher Külyutmaz.

- İnek Şaban ID: 1
- Güdük Necmi ID: 2
- Damat Ferit ID: 3
- Domdom Ali ID: 4
- Tulum Hayri ID: 5
- Hayta İsmail ID: 6, etc.



The data about the similarity of students' exam papers also requires clandestine storage methods. If students found a file containing only numbers between 0 and 100, they would certainly get suspicious and know it had something to do with the exam scores. Hence, Külyutmaz devised an encryption scheme: similarity scores will be encrypted in such a way that **the numbers at even positions in the array will be multiplied by 9, and the rest of the numbers will be divided by 3**. *"Let them try to crack this one!"* he chuckled, quite pleased with himself.

Furthermore, the encrypted similarity scores will be stored as a **row-major implementation of a lower triangular matrix** which holds the similarity scores between each pair of students as shown in table given below. E.g., $495/9 = 55$ is the encrypted similarity score between students 1 and 2 (İnek Şaban and Güdük Necmi).

Student IDs	1	2	3	4
1	#	#	#	#
2	495	#	#	#
3	10	864	#	#
4	22	0	24	#

Your mission is to tackle these problems one at a time, and detect cheaters. You will be given some C code to help you in the process.

Part 1: Arrays Using For Loops - Memory Access

The similarity scores should be defined as an array which will be stored in the memory. You can imagine it as illustrated below:

Oxaddr+14	24	<- A[n-1]
Oxaddr+10	0	
Oxaddr+C	22	
Oxaddr+8	864	
Oxaddr+4	10	<- A[1]
Oxaddr	495	<- A[0]

Memory before the execution.

Your program will first have to decrypt the similarity scores saved in the memory. The resulting actual similarity scores **must be saved in the same memory locations as the encrypted ones**. Consider the C code given below and write its MIPS equivalent.

```
for(int i = 0; i < datasize; i++){
    if(i % 2 == 0)
        data[i] = data[i] / 9;    // using div instructions is not allowed
    else
        data[i] = data[i] * 3;    // using mul/mult instructions is not allowed
}
```

Important constraints for this part: You are **not allowed to use multiplication and division instructions (mult, mul, div) for this problem**. Usage of those instructions will result in **penalty of -10 points**.

In MIPS an array A can be defined as illustrated below. Here, an array of six elements {495, 10, 864, 22, 0, 24} is declared in the memory, and the address of the first element is loaded to register t0.

```
.data
A: .word 495, 10, 864, 22, 0, 24  #Do not forget a space after a comma!
.text
.globl main
main:
    la $t0, A # Load the address of A[0] to register t0
```

After **la** instruction, the address of the first element of array A will be stored in register t0. You can write your remaining code afterwards. Note that the addresses of the next elements are increments by 4 as MIPS is byte-addressable. You can define the array size in the data segment as well.

After your code executes, the change in the memory should reflect as shown below. The before and after screenshots of the Data Segment in your simulator should be similar to the examples in figures below.

0xaddr+14	72	<- A[n-1]
0xaddr+10	0	
0xaddr+C	66	
0xaddr+8	96	
0xaddr+4	30	<- A[1]
0xaddr	55	<- A[0]

Memory after the execution.

Data Segment						
Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)
0x10010000	495	10	864	22	0	24

Data segment before the execution.

Data Segment						
Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)
0x10010000	55	30	96	66	0	72

Data segment after the execution.

Part 2: Function Calls

The vice-principal and history teacher Kel Mahmut, who was just as eager to catch any cheaters, suggested that they also count how many student pairs have a suspiciously high similarity score. This count would serve as an overall indicator of the cheating epidemic in the class. However, coding a simple loop-based counter would be too conspicuous, and would definitely catch the attention of possible intruders. So, Külyutmaz decided that it would be better to have a separate function which will perform this count recursively. This approach would definitely throw any prying student off. The recursive function to count how many elements in a given array of n numbers are greater than or equal to a given threshold T is defined as follows:

$$COUNT(A, n, T) = \begin{cases} 0 & \text{if } n = 0 \\ COUNT(A, n-1, T) + 1 & \text{if } A[n-1] \geq T \\ COUNT(A, n-1, T) & \text{otherwise} \end{cases} \quad (1)$$

Having this formula in mind, you should write another MIPS program to implement this recursive function that will operate on the decrypted array. The C code to which you may refer to is given below.


```
int count_suspicious(int *data, int n, int threshold){
    int count;
    if(n == 0)
        return 0;
    count = count_suspicious(data, n - 1, threshold);
    if(data[n - 1] >= threshold)
        count = count + 1;
    return count;
}
```

The final result of the recursive function should be stored in register *v1*, and output to the console. **Note that for the given test cases, the threshold value is 40.** The threshold should be defined as a variable in the data segment so that it can be changed for different test cases.

Important notes and constraints for this part:

- You must use **jal** instruction for function calls. You should also use registers *a0* – *a3* for passing arguments, and *v0* – *v1* for returning results, as is the convention in MIPS assembly.
- Note that this function has **three arguments** (*a0*: address of the array, *a1*: number of elements, *a2*: threshold). You must manage the stack carefully to preserve all necessary registers across recursive calls.
- For full credit you **MUST** implement the recursive function.
- You are allowed to use multiplication and division instructions (*mult*, *mul*, *div*) in this part.
- Implementing a function which is not recursive (one single call to a function that counts using a loop) will only get you 50% of credit for this part, provided you use the function call and stack properly. Otherwise 0 credit will be given.
- Counting using only a loop without any function calls will not get any credit either.
- You should include the comments in your code.

BONUS Part 3: Naming the Cheaters [EXTRA CREDIT]

If you wish to get extra credit, implement an additional functionality of detecting who the cheaters are. Assume that there will always be only two cheaters in each test case. **If the similarity score between two students is 80 or higher, then those two students are the cheaters.** Your program should store the cheater's IDs in registers *s6* and *s7* and output their names to the console. You may refer to the list of student IDs given at the beginning of the section (ID 1 is İnek Şaban, etc.).

For the sample input, the output should show that the cheaters are students 2 and 3 (*Güdük Necmi* and *Damat Ferit*), since their decrypted similarity score is 96.

As this is an extra credit part, you will not be given C code to refer to but you may keep the following in mind:

- The similarity scores are stored as a **row-major 1-D implementation of a lower triangular matrix**.
- You need to figure out the mapping between the 1-D array index and the (row, column) pair in the lower triangular matrix to determine which two student IDs correspond to the cheating pair.

Grading

- MIPS code: 100%
 - Part 1 - Memory (array) access using loops: 50 points
 - Part 2 - Function calls: 50 points
- Bonus part 3 - Find the cheaters: 10 bonus points

Sample Testcases

Test for the following testcases which you will include in your report, but your code will be tested with different testcases as well:

Testcase 1:

num_students = 4

datasize = 6

threshold = 40

data[] = {495, 10, 864, 22, 0, 24}

Expected output for testcase 1:

data[] = {55, 30, 96, 66, 0, 72}

count_suspicious (threshold=40) = 4

Bonus: Cheaters IDs: 2 and 3

The cheaters are G Necmi and Damat Ferit

Testcase 2:

num_students = 6

datasize = 15

threshold = 40

data[] = {90, 8, 405, 5, 180, 17, 324, 28, 108, 20, 189, 3, 378, 6, 270}

Expected output for testcase 2:

data[] = {10, 24, 45, 15, 20, 51, 36, 84, 12, 60, 21, 9, 42, 18, 30}

count_suspicious (threshold=40) = 5

Bonus: Cheaters IDs: 2 and 5

The cheaters are G Necmi and Tulum Hayri

Must-Use Starter Code

[Click to download the must-use starter code.](#)

How to Submit, What to Turn In & Deadline

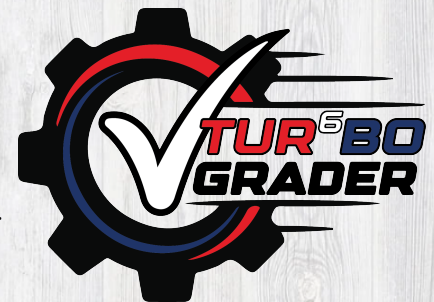
In this assignment, you will write and simulate a MIPS program as described in the instructions. Please turn in the following items in the specified format through the [submission system](#) <https://submit.cs.hacettepe.edu.tr/> (only .zip files are supported):

- **b<studentID>.zip**
 - cheaters.asm

Note that submissions via email or any other medium other than the [submission system](#) will **NOT be accepted**. The submission deadline is **Sunday, 05/04/2026 at 23:59:59**. Extensions or late submissions will **NOT be allowed**.

You should test your code via **Tur⁶Bo Grader** <https://test-grader.cs.hacettepe.edu.tr/> (but note that this does not count as submission!).

If you do not test your code via the grader before submission, you will not be able to object after the grades have been announced.



Plagiarism Control Notice

Students must implement their solutions individually. All submissions will be submitted to a plagiarism check. Any submissions that show a high level of similarity will be reported as plagiarism attempts to the ethics committee.